

Computer Science Extended Essay:

Investigating the use of machine learning in the determining of idle carbon emissions

Research Question:

- “To what extent can machine learning be applied to traffic lights to determine the amount of idle carbon emissions released?”

Word count: 3,820

Contents:

Introduction.....	3
Theory.....	3
Machine Learning.....	4
Vehicle Specific Power.....	5
K-Nearest Neighbor Algorithm.....	6
Encoding.....	8
Comparison of One Hot and Label Encoders.....	8
The Process.....	9
Importing the Data.....	9
Calculating VSP and Carbon Emissions.....	10
Splitting the Data.....	12
Building and Applying the model.....	12
Analyzing the Predictions.....	13
Data Correlation.....	14
Implications of Findings.....	15
Limitations of Investigation.....	16
Conclusion.....	16
Appendix.....	18
References.....	19

Introduction:

The purpose of this essay is to explore the application of **machine learning** to traffic signals and determine the amount of unproductive carbon emissions. This essay focuses on basic supervised learning, which is a subset of machine learning in which an algorithm maps all inputs to an output, based on example input-output pairs that were used to train the model. It essentially allows a software-defined agent to predict outputs based on labelled input values provided to it. Here, the aim is to make sense of data toward specific measurements. The issue of global warming is a dark shadow that looms over the world and is constantly being fought to preserve the planet and every little step counts. Studies show that about 25% of drivers' exposure to harmful carbon emissions is at traffic lights that only make up about 2% of their total travel time every day. The waiting time at the signal would be directly linked to the idle carbon emissions given off by cars. So, the less people wait, the less unproductive carbon is released into the air. Hence, the question: *To what extent can machine learning be applied to traffic lights to determine the amount of idle carbon emissions released?*

This is, by no means, a new issue or field of research to the governments of the world. The research to solve this problem is still ongoing, and officials are understandably reluctant to release word of their progress and datasets. So, the data used in this analysis is data found on github.com and unfortunately repetitive, but I found that applying an encoder and increasing the timeframe for which this data was acquired, further complicated the model's algorithm.

Theory:

Machine Learning:

Machine learning is used to teach a computer to make conclusions of its own based off of patterns found in massive amounts of data fed to it. The key difference between regular computer software and machine learning is that the coder doesn't teach the computer how to come to conclusions, but only provides it with large amounts of reference data to understand, differentiate between and recognize patterns. Machine Learning is used to perform tasks that humans have been doing manually till now, especially tasks that humans use their innate intelligence to perform. The purpose of Machine Learning is to replicate this intelligence.

Machine Learning has 7 critical steps.

1. **Gathering data:** Understanding the caliber and intricacy of the data required shows how appropriate and intricate the model (environment) to be created will be.
2. **Data preparation:** The data is to be prepared by disputing (to prepare it for training), cleaning, randomizing and visualizing it. Finally, the data is to be split into three sets: training, evaluation and test.
3. **Choosing a model:** The right model can only be built or picked by accurate collection and preparation of data, because real life data tends to contain underlying mathematical models and the point of machine learning is to uncover these models. Without the right algorithm, the accuracy of the prediction will be greatly affected.
4. **Training:** In the process of training, the aim is for the machine to acquire the correct answer/prediction as often as possible. For this stage, the training set of data will be used.
5. **Evaluation:** This step is used to “measure” the performance of the machine against a unit or a combination of units. This is done by testing the machine against data that hasn’t been seen before. In this, the evaluation set of data is to be used.
6. **Hyperparameter Tuning:** This stage is used to fine tune the existing parameters so as to fix the issues found in the previous stage of evaluation. A few of these problems could be the number of steps required for training or even the rate of learning.
7. **Prediction:** Introduce the test data that has been withheld from the machine till now to further test the model and understand how well the model would work in the real world.

These steps are almost identical to every type of Machine Learning, except for the data preparation and training stages. There are three types of Machine Learning: supervised learning, unsupervised learning and reinforcement learning. Supervised learning, being the most common, prepares data in a way that it is labelled, so that in the training stage, the machine already knows the patterns it is supposed to identify. In unsupervised learning, the data is left unlabeled. So, in the training stage, the machine notices common factors in collections of data, identifies patterns and thus arranges the data. In reinforcement learning, there is no predefined data and the agent follows the trial and error method to work in accordance with a reward system where it receives a reward for a correct prediction and a penalty for an incorrect prediction.

This essay will be focusing on the functionality of a supervised machine learning model.

In the process of supervised learning, there is a dependent variable (y) and an independent variable (x). In this branch of machine learning, all data provided to the agent is labelled and organized. The goal of the algorithm of a supervised learning model is to map the output $[f(x)]$ directed from each new value of the independent variable (x), based on the data sets provided to the agent. We can think of it in the sense that the training data set provided to the agent supervises or guides its prediction of the testing data.

Supervised Machine Learning, Algorithms are classified on the basis of the output produced, are:

1. Regression Algorithms

Given an input (x) Regression Algorithms use a mapping function $f(x)$ and attempt to estimate or predict a continuous or numerical variable (y), where y is an Integer or Floating-point, Real value.

Typical outputs of a Regression Algorithm would be abstractions of “Mass”, “Volume”, etc. which are Real, Quantitative attributes.

2. Classification Algorithms

Given an input (x), Classification Algorithms use a mapping function $f(x)$ and attempt to estimate or predict a discrete or categorical value (y), where y is a Class value. Typical outputs of a Classification Algorithm would be abstractions of categories such as “Pollutant” or “Non-Pollutant”, etc. which are Qualitative attributes

Given the fact that this essay will be focusing on the predicted value of carbon emissions released at a traffic signal, a model with a regression algorithm will be used.

Vehicle Specific Power

The theory of Vehicle Specific Power is a key factor in the calculation of the carbon emissions released. It is the estimate of the power demanded on the engine while driving the vehicle. It is calculated in kW/Metric Ton, using the second-by-second speed values in a driving schedules, along with information about the vehicle being operated. The calculation of VSP reached maximum efficiency and convenience, in the sense that it is a one-dimensional measurement and can be calculated using only roadside measurements (mass is only used in the calculations for the aerodynamic term, but it is usually ignored in the real world application on roads), and it allows a direct physical interpretation of the data collected.

Of course, the VSP and emission values differ not only according to the model of the vehicle but also with the type of fuel used. To calculate VSP, the speed, acceleration and physical factors of the road must also be taken into consideration, in accordance with the following formula, applicable to most light-duty vehicles.

$$\text{VSP} \left[\frac{\text{W}}{\text{kg}} \right] = \frac{\text{Power}}{\text{Mass}} = \frac{\frac{d}{dt}(E_{\text{kinetic}} + E_{\text{potential}}) + F_{\text{rolling}} * v - F_{\text{aerodynamic}} * v}{m}$$

Here, it can be seen that the most fundamental version of the formula is simply the power exerted by the engine by the mass of the vehicle. The calculation of the power is also inherently basic. It is simply the gradient function of the total energy of the vehicle (kinetic energy + potential energy) added to the rolling resistance force exerted by the road times the instantaneous speed of the vehicle minus the aerodynamic resistance force (or drag force) times the instantaneous speed.

The rolling resistance force and aerodynamic resistance force are factors of VSP that are affected by conditions such as the quality or grade of the road, wind speed, etc.

K-Nearest Neighbor Algorithm

The K-Nearest Neighbor (KNN) algorithm is a simple algorithm used in supervised learning. It is used to solve both, classification and regression problems. This analysis deals with a regression problem. The KNN algorithm bases its analysis of data predictions on its assumption that similar data points or observations exist close together. We can say that it functions by the principle of “birds of a feather flock together”

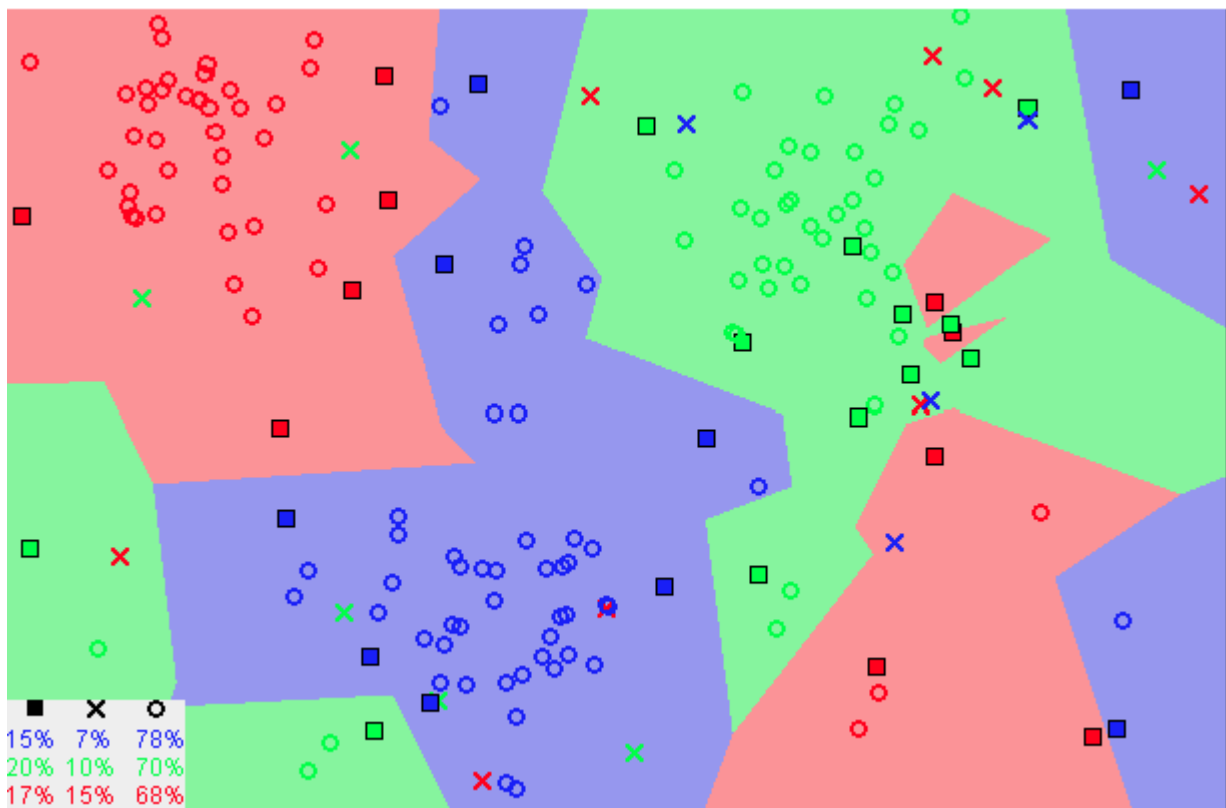


Figure 1: Figure showing how similar data points typically exist close to each other

The KNN algorithm depends on the assumption that similar data point being close to each other being true enough for the algorithm to be useful. KNN captures the idea of similarity or proximity through the basic mathematical process of measuring the distance between two data points if they were to be represented on a graph. So, the process of functioning of the KNN algorithm, on paper, is relatively simple. Once the data is loaded into the algorithm, the K number (the number of centroids/similar data sets the algorithm should be looking for) must be initialized to the chosen number of neighbors. Now, for each example of the data, the distance is measured from the current example and the query example in the data, and the distance and index are added to an ordered collection. The collection is meant to be ordered so that the distances and indices can be sorted in ascending order (according to distance) for the next step of the process, that is, the picking of the first K entries in the sorted collection. Then the labels (values) of the K entries have to be acquired. The next step depends on the type of model (classification or regression) being used. If it is a classification, the algorithm provides the output as the mode (the most frequent value) of all the values of the K entries and if it is a regression, the output or the prediction is given by calculating the mean (the average) of all the values of the K entries.

To select the right value of the K number, I ran the KNN algorithm multiple times to find the right number that reduced the errors while maintaining the algorithm's ability to accurately make predictions on data it hasn't seen before. As the K number I chose got closer to 1, the predictions became more unstable. However, the increment of the K number had the inverse effect in the sense that the predictions became more stable the more neighbors it was allowed to analyze. However, this was only to a certain point. Eventually, I began to notice that the number of errors began to increase as I increased the K number, showing that I had pushed the K number too far. I eventually found the optimal balance of the K number in the value 127.

Encoding (One Hot Encoder)

Often, datasets will include values that are text based, unlike the numerical values that are easier to read and understand in the process of machine learning. Regardless of what the value is, the challenge here would be to determine how exactly the data is to be used in the analysis. So, the purpose of encoding is to convert the categorical data into useful numerical data. Many machine learning algorithms are capable of using textual data without using an encoder. However, the algorithm used in this analysis is a K-Nearest Neighbor algorithm that automatically ignores the categorical data and only uses numerical data.

There is a large variety of encoders available in python libraries, but the One Hot encoder is used in this analysis. The functionality of a One Hot encoding is fundamentally very simple. Here, it takes a column which has been label encoded and then splits the single column up into multiple columns. The values of the single column each acquire a column of their own, with values of 1s and 0s to indicate its presence or absence in the particular occurrence.

Comparison of One Hot and Label Encoder

Another encoder that could have been used is a Label Encoder. In a Label encoder, all that would've had to be done is convert/encode the non-numerical values of the first column into machine readable numerical values. This would be done by importing the "LabelEncoder" class from the sklearn library, fit and transform the data in the first column and then replace the existing text data with the new encoded data. However, the problem I came across here was that the model might mistake the numerical replacements for the text to have

some sort of numerical hierarchy ($0 < 1 < 2$) and arrange or treat the text in that manner. To overcome this barrier, the OneHot Encoder has been used.

In the datasets imported to the coding site, the information as to what route was taken by the vehicle has also been provided. Originally, these were categorical values, but to allow the inclusion of these values in the data analysis, they had to be converted to numerical values. Using the One Hot encoder, each possible categorical value was converted to a column in the dataset, and used binary (1s and 0s) values as a true or false value for the route taken by that particular vehicle.

```
[29]: encoder = ce.OneHotEncoder(cols=['route1'],return_df=True,use_cat_names=True)
df = encoder.fit_transform(df)
```

```
[31]: df.head()
```

```
[31]:
```

	interval	startTime	endTime	route1_road_1_2_3	route1_road_2_1_2	route1_road_0_1_0	route1_road_1_0_1	route1_road_0_1_1	route1_road_2_1_3	route1_road_1_0_2	...	maxPc
0	5	1	1	1	0	0	0	0	0	0	...	
1	5	16	16	0	1	0	0	0	0	0	...	
2	5	22	22	0	1	0	0	0	0	0	...	
3	5	24	24	0	1	0	0	0	0	0	...	
4	5	27	27	0	0	1	0	0	0	0	...	

5 rows × 68 columns

Figure 2: Figure showing the process of encoding the values and final encoded values

The Process:

Importing the data

To carry out this investigation, it was first necessary to import the data, sourced from github.com, to the Python Notebook file on JupyterLab to make the data accessible to the algorithm being used to calculate and predict the VSP values. The data is stored in sets of rows and columns and is derived from recording 23204 vehicles passing through one intersection across multiple different routes within a time period of 12 hours (8AM – 8PM). In order to correctly read and interpret the data, one would need a data dictionary (see appendix). The secondary data which this entire process is built on had to be modified as the original values were duplicated throughout the data frames, so the columns “startTime” and “endTime” were added, along with the randomized values of the column “idling” (the amount of time the vehicle was still for) that lie between 10 and 30 seconds.

Calculating VSP and Carbon Emissions

The values of VSP that were to eventually be predicted were calculated with the following function:

$$\text{vsp} = \text{speed} * (\text{a} * \text{accel} + (\text{g} * \text{slope}) + \text{b}) + (\text{c} * \text{speed}^3)$$

By default: a = 1.1, b = 0.132, c = 0.000302 and g = 0.132 (as of Jimenez-Palacios, 1999).

The values of the VSP were calculated as they too were not present in the original dataset, but are an integral part of this analysis.

```
[43]: def vsp(speed, accel):  
      return math.floor(speed * ((1.1 * accel) + (.132) + 0.132) + (0.000302 * (speed**3)))  
  
[44]: df["vsp"] = df.apply(lambda x: vsp(x['maxSpeed'], x['usualPosAcc']), axis=1)
```

Figure 3: Figure showing how VSP was calculated

To further better the application of the model, data frames for the VSP modes and Fuel consumption averages were created. As mentioned before, the amount of carbon emitted is largely affected by the type of fuel being used by the vehicle, hence the importance of the inclusion of the values in the dataset. The Instantaneous Fuel Consumption display shows the value of instantaneous fuel consumption (the kms per litre your car is getting right at that very moment when your car is moving). The display shows estimated values.

```
[45]: vsp_modes = pd.DataFrame({'vsp_mode': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14],  
                              'wkg_low': [-5, -2, 0, 1, 4, 7, 10, 13, 16, 19, 23, 28, 33, 39],  
                              'wkg_high': [-3, -1, 0, 3, 6, 9, 12, 15, 18, 22, 27, 32, 38, 40]})  
  
[46]: fuel_consumption = pd.DataFrame({'vsp_mode': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11],  
                                       'petrol(gm/sec)': [.33, .5, .71, 1, 1.2, 1.35, 1.48, 1.65, 1.8, 1.85, 1.9]})
```

Figure 4: Figure showing how the VSP mode and Fuel Consumption data frames were created

The values of these data frames were influenced by researched VSP modes and instantaneous fuel consumption values for gasoline and diesel.

VSP mode	Instantaneous fuel consumption (l)	
	GASOLINE	DIESEL
1	0.01244	0.01116
2	0.01866	0.01674
3	0.020526	0.018414
4	0.0622	0.0558
5	0.08397	0.07533
6	0.11507	0.10323
7	0.14306	0.12834
8	0.16794	0.15066
9	0.19904	0.17856
10	0.22703	0.20367
11	0.27368	0.24552

Figure 5: Table showing the researched instantaneous fuel consumption (in litres) for different types of fuels and VSP modes

To form the crux of the analysis, it is obviously integral to calculate the emissions given off by each individual car. For a light vehicle, approximate fuel consumption during idling is 1.89 litres per hour. This is calculated as 3.93 grams of carbon emission per second. The mass of 1 litre volume of gasoline is 750gms. Since gasoline is 87% carbon, 1 litre contains 652g of carbon. For full oxidation of this carbon to CO₂, 1740g of Oxygen is required. The mass of CO₂ per litre of gasoline therefore is (652+1740) = 2392g per litre of gasoline.

```
[49]: def co2_emissions(petrol,idle):
      co2 = (petrol + (3.93*idle)) *.87 * 2.66
      return co2
```

Calculate Column for CO2 Emissions

```
[50]: final["CO2/ltr"] = final.apply(lambda x: co2_emissions(x["petrol(gm/sec)"],x["idling"]),axis=1)
```

Estimated Total Emissions during a 60 Minute Period

```
[51]: print(f'{round(final["CO2/ltr"].sum()/1000,2)}kg CO2')
4215.82kg CO2
```

Figure 6: Figure showing calculation and inclusion of CO₂ emissions in dataset

Splitting the Data

To allow the model to work, we would have to test it. So, the data is to be split into two sections: Training data and Testing data. A larger percentage of the data will be used to train the model than to test its functionality. The data is to be split according to its dependent and independent variables, which are as follows:

Independent variables: Time, idling, route taken

Dependent variables: CO₂ per unit litre

```
[77]: X = final[['24hour', 'idling', 'route1_road_1_0_8', 'route1_road_1_2_6']]
      y = final[['CO2/ltr']]
      from sklearn.model_selection import train_test_split

      X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=100)
```

Figure 7: Figure showing the split of data into training and testing datasets

Building and Applying the model

Once the data has been split, it must be applied to the model to be analyzed and used for the training stage so as to be able to make predictions. As mentioned earlier, this particular model uses a K-Nearest Neighbor Regression algorithm to identify feature similarities and predict the values of new data. Here, the algorithm found 127 neighboring values for each data point to base its predictions on.

```
[78]: from sklearn.neighbors import KNeighborsRegressor

      model = KNeighborsRegressor(n_neighbors=math.floor(math.sqrt(len(X_train))))

      model.fit(X_train, y_train)

[78]: KNeighborsRegressor(n_neighbors=127)
```

Figure 8: Figure showing use of K-Nearest Neighbor

To begin the application of the model 23,204 rows and 72 columns of data were fed into the algorithm to acquire a prediction of the CO₂ emissions per litre of fuel.

```
[79]: y_predict = model.predict(X_test)
```

Figure 9: Figure showing how the model was applied

To check the accuracy of the model, it was first applied to the training data itself. Here, it should get a high accuracy percentage because the data we've trained it with is the same data it is getting tested on, so ideally the accuracy percentage should be 100%, but due to foreseen uncertainties and errors, the accuracy percentage acquired was 99.8%.

```
[81]: f'{round(model.score(X_train,y_train) * 100,2)}%'  
[81]: '99.8%'
```

Figure 10: Figure showing application of model on training data and accuracy percentage

Now that it has been proven that the accuracy of the algorithm and the model is sufficiently high, the model is applied to the testing data to predict values it has never seen before, unlike the previous testing stage where the data that the model was predicting was the same as the data it had been trained with. Typically, the accuracy percentage should be lower for the predictions on the testing data than those on the training data. Hence, the accuracy percentage acquired on the testing data is 99.79%.

```
[80]: f'{round(model.score(X_test,y_test) * 100,2)}%'  
[80]: '99.79%'
```

Figure 11: Figure showing the application of the model on testing data and corresponding accuracy percentage

Analyzing the Predictions

Once the predictions have been made, a new data frame would have to be created to view the predicted data versus the actual data.

```
[82]: pred = y_test.copy()
      pred.insert(1,"Prediction",y_predict)

[83]: pred.rename(columns={"Predicted":"Actual"},inplace=True)

[88]: pred
```

	CO2/ltr	Prediction
9624	95.345040	97.135356
6668	186.293100	186.436325
18113	249.956742	246.734173
13743	95.345040	97.565032
15721	231.767130	234.345185
...	...	...
8136	149.913876	148.051947
7930	95.345040	97.135356
21996	268.146354	265.353461
18709	149.913876	145.903568
2079	149.913876	146.906145

6961 rows × 2 columns

Figure 12: Figure showing creation of new data frames and tabular representation of comparison of actual and predicted CO2/litre emissions

Data Correlation:

In order to rationalize the reasoning behind the predictions made by the model, the independent and dependent variables were compared to find a correlation between the two. This was done through the use of a heat map where it can be seen that the correlation between each variable has been given a numerical value between 0 and 1. The higher the correlation, the higher the numerical value in each grid, and the lighter the color of the grid.

[76]: <matplotlib.axes._subplots.AxesSubplot at 0x18004d1ed30>

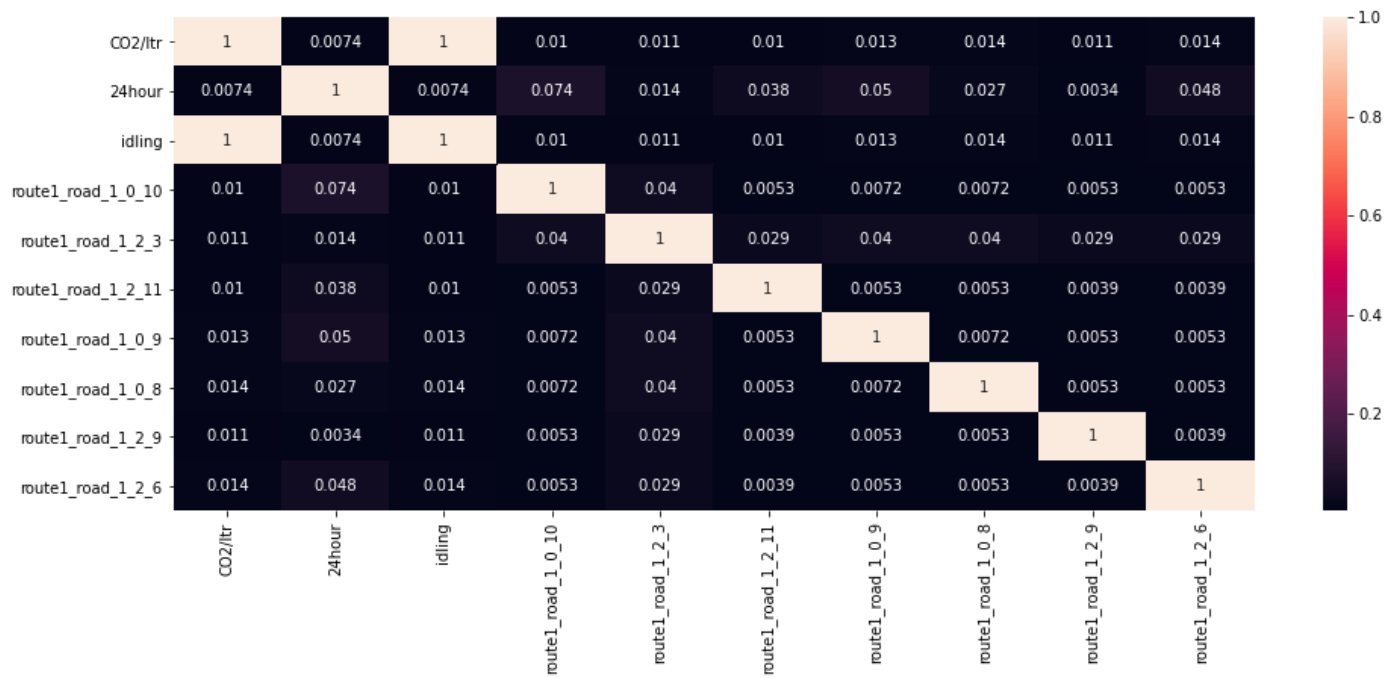


Figure 13: Grid to show correlation between variables

Limitations of the Investigation:

The limitations of the investigation include the following:

1. The data found on the internet was extremely repetitive and had to be modified in order to better train the model and not acquire a 100% prediction rate on the testing as well as training data. This is due to the previously mentioned fact that governments haven't released their findings on the study of the same topic because it is a matter of national and eventually global importance.
2. Another limitation of the analysis was the lack of correlation in the data, allowing minimal data analysis.
3. Additionally, the repetitiveness of the data also didn't leave much room for the identification of error bars, and once more, further data analysis.

Implications of Findings:

The findings of this investigation show that machine learning is a plausible method of calculating and predicting idle CO₂ emissions at traffic signals, given a sufficient amount of training and testing data. Governments and research organizations have spent millions of dollars on this investigation, but no solution

to optimize route arrangement and/or idling time has been found to be of significant use to road and transport authorities. However, the correlations that have been established can be used. Fuels such as petrol and diesel increase VSP and hence, CO₂/litre emissions. Additionally, the instantaneous speed of the vehicle immediately before and after stopping at the traffic light are significant factors in the production of carbon emissions. In layman's terms, the faster the vehicle was going before it stopped at the junction, the more work and power it would exert on the engine to stop. Once again, if the vehicle is made to start at a high speed immediately after stopping, it would take a lot more work and power demanded from the engine to acquire that speed in a short period of time.

Understanding the significance of the role traffic signals play in the contribution to air pollution today in the modern world is another substantial step towards net zero carbon emissions. If one were to look at CO₂ emissions in the g/km fraction, one would argue that more emphasis should be put on the mechanics of how each kilometer is traveled, including routes taken, traffic control signage in place, and so on. Studies like my own help further research on in-vehicle intelligent transport systems (ITS) and even smart signals leading to significant improvements in terms of environmental impact, by optimizing route selection and idling time. So, being able to evaluate, quantify and apply the impact of technology in terms of carbon emissions is very important and eventually leads to the integration in the assignment value of emissions of new cars, which is relevantly reflected in the final price/yearly taxes and can contribute to a more rapid dissemination of such technologies.

Conclusion:

This investigation sought to find the extent to which a linear regression supervised machine learning algorithm can predict the amount of unproductive carbon emissions released at a traffic signal. These calculations are based off the idling time, through the use of the vehicle specific power formula and mode values to show the amount of carbon emitted per litre for different types of fuel.

The first version of the model provided a 100% prediction rate due to the repetitiveness of the data. Although, this established a very important fact that the engine was correctly modelled and fully capable of recognizing patterns and understanding probabilities with the correct use of the K-Nearest Neighbor algorithm.

To overcome this limitation, the additional use of an encoder was deployed to provide the model with more input variables namely, the route taken by each vehicle at the traffic signal, for the algorithm to be applied to, further enriching the process. This resulted in a more realistic prediction percentage in the final draft of the code.

In conclusion of the experimentation and analysis of the result: a linear regression supervised machine learning model has been proven to be able predict the carbon emissions at traffic signals. This conclusion stands as reliable despite all the data constraints of the model. However, the analysis could be far more realistic if the data itself was more representative of real-life conditions. The correlation between carbon emissions and the idling time of the car is an important insight that governments should be taking seriously as a part of the process to reduce their carbon footprint by contributing to the research of Smart Traffic Signaling Systems, Dynamic Timing, Congestion Control, etc. This investigation also serves as an example of the vast possibilities of prediction using linear regression as a supervised machine learning algorithm and shows the opportunities Artificial Intelligence can explore in the real world.

APPENDIX:

Data dictionary:

- length: length of the vehicle
- width: width of the vehicle
- maxPosAcc: maximum acceleration (in m/s)
- maxNegAcc: maximum deceleration (in m/s)
- usualPosAcc: usual acceleration (in m/s)
- usualNegAcc: usual deceleration (in m/s)
- minGap: minimum acceptable gap with leading vehicle (in meter)
- maxSpeed: maximum cruising speed (in m/s)
- headwayTime: desired headway time (in seconds) with leading vehicle, keep current speed * headwayTime gap.

REFERENCES:

Bibliography:

Reinforcement Learning for Traffic Signal Control, traffic-signal-control.github.io/.

“Supervised vs Unsupervised vs Reinforcement Learning: Intellipaat.” *Intellipaat Blog*, 19 Oct. 2020, intellipaat.com/blog/supervised-learning-vs-unsupervised-learning-vs-reinforcement-learning/.

Brownlee, Jason. “Supervised and Unsupervised Machine Learning Algorithms.” *Machine Learning Mastery*, 20 Aug. 2020, machinelearningmastery.com/supervised-and-unsupervised-machine-learning-algorithms/.

Kumar, Prashant, and University of Surrey. “Why Traffic Lights Are Pollution Hotspots.” *World Economic Forum*, www.weforum.org/agenda/2015/02/why-traffic-lights-are-pollution-hotspots/.

Brownlee, Jason. “Supervised and Unsupervised Machine Learning Algorithms.” *Machine Learning Mastery*, 20 Aug. 2020, machinelearningmastery.com/supervised-and-unsupervised-machine-learning-algorithms/.

“Category Encoders¶.” *Category Encoders - Category Encoders 2.2.2 Documentation*, contrib.scikit-learn.org/category_encoders/.

“Category Encoders¶.” *Category Encoders - Category Encoders 2.2.2 Documentation*, contrib.scikit-learn.org/category_encoders/.

Chen, Zhong, et al. “Estimating Fuel Consumption and Emissions Based on Reconstructed Vehicle Trajectories.” *Wiley Online Library*, John Wiley & Sons, Ltd, 20 Nov. 2012, onlinelibrary.wiley.com/doi/pdf/10.1002/atr.1215.

Chen, Zhong, et al. “Estimating Fuel Consumption and Emissions Based on Reconstructed Vehicle Trajectories.” *Wiley Online Library*, John Wiley & Sons, Ltd, 20 Nov. 2012, onlinelibrary.wiley.com/doi/pdf/10.1002/atr.1215.

“Flow File Format¶.” *Flow File Format - CityFlow 0.1 Documentation*, cityflow.readthedocs.io/en/latest/flow.html.

Garbade, Dr. Michael J. “Regression Versus Classification Machine Learning: What's the Difference?” *Medium*, Quick Code, 11 Aug. 2018, [medium.com/quick-code/regression-versus-classification-machine-learning-whats-the-difference-345c56dd15f7#:~:text=Regression and classification are categorized,umbrella of supervised machine learning.&text=The main difference between them,is categorical \(or discrete\).](https://medium.com/quick-code/regression-versus-classification-machine-learning-whats-the-difference-345c56dd15f7#:~:text=Regression and classification are categorized,umbrella of supervised machine learning.&text=The main difference between them,is categorical (or discrete).)

Garbade, Dr. Michael J. “Regression Versus Classification Machine Learning: What's the Difference?” *Medium*, Quick Code, 11 Aug. 2018, [medium.com/quick-code/regression-versus-classification-machine-learning-whats-the-difference-345c56dd15f7#:~:text=Regression and classification are categorized,umbrella of supervised machine learning.&text=The main difference between them,is categorical \(or discrete\).](https://medium.com/quick-code/regression-versus-classification-machine-learning-whats-the-difference-345c56dd15f7#:~:text=Regression and classification are categorized,umbrella of supervised machine learning.&text=The main difference between them,is categorical (or discrete).)

Reinforcement Learning for Traffic Signal Control, traffic-signal-control.github.io/.

“Supervised vs Unsupervised vs Reinforcement Learning: Intellipaat.” *Intellipaat Blog*, 19 Oct. 2020, intellipaat.com/blog/supervised-learning-vs-unsupervised-learning-vs-reinforcement-learning/.

Ferreira, Michel & d'Orey, Pedro. (2012). On the Impact of Virtual Traffic Lights on Carbon Emissions Mitigation. *IEEE Transactions on Intelligent Transportation Systems*. 13. 284-295. 10.1109/TITS.2011.2169791.

Harrison, Onel. "Machine Learning Basics with the K-Nearest Neighbors Algorithm." *Medium*, Towards Data Science, 14 July 2019, towardsdatascience.com/machine-learning-basics-with-the-k-nearest-neighbors-algorithm-6a6e71d01761.